

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. ОБЩАЯ ХАРАКТЕРИСТИКА ВЕБ-ПРИЛОЖЕНИЯ-АГРЕГАТОРА УСЛУГ	6
1.1. Базовые понятия веб-разработки и агрегаторов услуг	6
1.2. Технологический стек: Django, HTMX, Alpine.js, Tailwind CSS	8
1.3. Принципы функционирования веб-приложения и динамического поиска 10	
ГЛАВА 2. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ БАРА «ПРИБОЙ»	14
2.1. ER-модель базы данных	14
2.2. Создание подсистем (Django-приложений)	17
2.3. Справочники и вспомогательные сущности	19
2.4. Основные сущности заказа и бронирования	22
2.5. Реализация системы динамического поиска	29
2.6. Клиентский интерфейс оформления заказа и бронирования	32
2.7. Административная панель и отчёты (Jazzmin)	37
2.8. Пользователи и система аутентификации	42
ЗАКЛЮЧЕНИЕ	44
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	45

ВВЕДЕНИЕ

Общественное питание традиционно консервативно в цифровизации: многие заведения по-прежнему работают по телефону и бумажным меню. Однако после 2020 года стала очевидна уязвимость бизнеса без цифровых каналов. Бар «Прибой», на базе которого выполнялась практика, не имел единого онлайн-канала: меню публиковалось в соцсетях, бронирование - по телефону, акции анонсировались разрозненно. Это вело к потере обращений и нагрузке на персонал. Создание веб-приложения-агрегатора, объединяющего меню, бронирование столов с кальянами и предзаказ на вынос, является актуальной задачей.

Цель выпускной квалификационной работы - разработка веб-приложения агрегатора услуг, включающая проектирование базы данных и реализацию системы динамического поиска.

Для достижения поставленной цели сформулированы пять задач:

1. Выбрать и объяснить, зачем нужны Django, HTMX, Alpine.js, Tailwind и Jazzmin.
1. Сделать базу данных в 3НФ, написать модели и миграции.
2. Сделать форму заказа и бронирования (модальное окно с двумя вкладками).
3. Настроить админку для управления меню, заказами и бронями.
4. Сделать поиск по меню с задержкой запросов и обновлением результатов.

Объект исследования - процессы обслуживания посетителей бара «Прибой» (в зале и на вынос).

Предмет исследования - средства разработки веб-приложения-агрегатора: Django, HTMX, Alpine.js, Tailwind CSS, тема админки Jazzmin.

ГЛАВА 1. ОБЩАЯ ХАРАКТЕРИСТИКА ВЕБ-ПРИЛОЖЕНИЯ-АГРЕГАТОРА УСЛУГ

1.1. Базовые понятия веб-разработки и агрегаторов услуг

Веб-приложение представляет собой программу, исполняющуюся на удалённом сервере и доступную пользователю через браузер по сети Интернет с использованием протокола HTTP или его защищённой версии HTTPS. В отличие от настольных приложений, веб-приложения не требуют установки на устройство клиента и могут открываться на любой платформе, имеющей современный веб-браузер. Это упрощает обслуживание программного продукта: обновления выпускаются на сервере и моментально становятся доступными всем пользователям без необходимости рассылать новые версии.

Архитектура любого веб-приложения строится на клиент-серверной модели взаимодействия. Клиент (браузер) отправляет HTTP-запрос на сервер, сервер обрабатывает запрос и возвращает HTTP-ответ, как правило содержащий HTML-документ, изображение, JSON-объект или иной ресурс. Современный HTTP-обмен ведётся по защищённому каналу HTTPS, использующему шифрование TLS, что предотвращает перехват и подмену передаваемых данных.

Под агрегатором услуг понимают веб-приложение, объединяющее в одной точке доступа разнородные предложения от одного или нескольких поставщиков. От обычного каталога агрегатор отличается возможностью совершать на сайте целевые действия (заказать, забронировать, оплатить), а от маркетплейса - отсутствием множественности конкурирующих поставщиков и наличием единого хозяина услуг. Применительно к предприятию общественного питания агрегатор позволяет совмещать просмотр меню, оформление заказа на вынос и бронирование столиков, что снимает нагрузку с телефонной линии и сокращает время обслуживания клиента.

База данных является основой любого современного веб-приложения. В подавляющем большинстве случаев используется реляционная модель: данные хранятся в виде таблиц, состоящих из строк и столбцов, а между таблицами устанавливаются связи через первичные и внешние ключи. Чтобы избежать избыточности и аномалий при изменении данных, схему доводят до третьей нормальной формы (3НФ): каждая таблица имеет первичный ключ, все неключевые атрибуты функционально зависят только от ключа и не зависят транзитивно от других неключевых атрибутов. Для ускорения операций поиска и сортировки на часто запрашиваемые поля создают индексы.

Под динамическим поиском в контексте веб-приложений понимают механизм, при котором поисковая выдача обновляется на странице без полной перезагрузки. Технически это реализуется через асинхронные HTTP-запросы (AJAX): пользователь набирает текст в поле ввода, скрипт после короткой паузы (дебаунса 200-400 мс) отправляет запрос на сервер, получает результат и подменяет содержимое заданного блока страницы. Такой подход сокращает количество кликов и улучшает воспринимаемую скорость работы сайта, что является значимым фактором удержания пользователей в современных интерфейсах.

Бар «Прибой» предоставляет посетителям комплекс услуг: меню кухни и бара, отдельный кальянный ассортимент, столики разной вместимости, регулярно обновляющиеся акции. До настоящего момента эти данные поддерживались разрозненно: меню в распечатанном виде, бронирование - по телефону, акции - в социальных сетях. Объединение их в единое веб-приложение с возможностью самостоятельного оформления заказа и бронирования столика позволит уменьшить операционную нагрузку на персонал и одновременно повысить удобство для посетителей.

1.2. Технологический стек: Django, HTMX, Alpine.js, Tailwind CSS

Для реализации поставленной задачи был выбран стек технологий, основанный на одном из наиболее зрелых веб-фреймворков языка Python - Django - и трёх лёгких клиентских библиотеках: HTMX, Alpine.js и Tailwind CSS. Такой выбор позволяет получить полнофункциональное приложение без необходимости разрабатывать отдельный клиентский SPA-фронтенд на React или Vue, что существенно сокращает трудоёмкость и упрощает поддержку.

Django - это серверный веб-фреймворк, реализующий шаблон проектирования Model-View-Template (MVT). Модели описывают сущности предметной области и сохраняются в базе данных через встроенный объектно-реляционный отображатель (ORM); представления (views) обрабатывают входящие HTTP-запросы и формируют ответы; шаблоны (templates) на основе HTML с расширенным синтаксисом превращают данные в готовый ответ для клиента. Django из коробки предоставляет систему миграций, административную панель, систему аутентификации, защиту от наиболее распространённых веб-атак (CSRF, XSS, SQL-инъекции) и базовые средства интернационализации.

HTMX - клиентская библиотека, которая расширяет стандартный HTML атрибутами, позволяющими выполнять асинхронные HTTP-запросы и подставлять полученный HTML-фрагмент в указанный участок страницы. К основным атрибутам относятся «hx-get» и «hx-post» (тип HTTP-запроса), «hx-target» (куда вставлять ответ), «hx-swap» (как именно подставлять), «hx-trigger» (по какому событию инициировать запрос), «hx-indicator» (какой элемент показать на время загрузки). Атрибут «hx-boost» превращает обычные ссылки и формы в HTMX-запросы, обеспечивая SPA-подобное поведение без полной перезагрузки страницы. Подход называется hypermedia-driven, поскольку обмен между клиентом и сервером происходит на уровне HTML, а не JSON.

Alpine.js - это компактная клиентская библиотека (около 10 килобайт), предоставляющая декларативную реактивность непосредственно в HTML-разметке. Основными директивами являются «x-data» для объявления локального состояния, «x-show» и «x-if» для условного отображения, «x-model» для двустороннего связывания, «x-on» (или сокращение «@click») для обработчиков событий, «x-transition» для CSS-анимаций. Помимо локальных компонентов Alpine.js поддерживает глобальные хранилища (Alpine.store), что удобно для управления общим состоянием - например, для корзины заказа, открытия модальных окон или очереди всплывающих уведомлений.

Tailwind CSS - это библиотека стилей, реализующая подход utility-first, в котором готовые CSS-правила сгруппированы в маленькие классы (например, «flex», «gap-4», «text-sm», «bg-violet-500»). Дизайн собирается прямо в разметке из этих утилит, без написания собственных CSS-классов для каждого компонента. В работе использована четвёртая мажорная версия библиотеки с новой системой конфигурации темы через директиву «@theme», размещаемую прямо в основном CSS-файле. Tailwind компилируется в production-сборку через CLI, отдавая в браузер только реально используемые правила, что обеспечивает минимальный размер итогового файла стилей.

Для административной панели применяется пакет Jazzmin, заменяющий стандартный шаблон Django Admin на современный интерфейс на базе Bootstrap 4. Jazzmin поддерживает тёмный режим (тема «darkly»), кастомизируемые иконки на основе FontAwesome, фиксированные боковую панель и верхнюю панель, что особенно важно для оператора, работающего одновременно с большим количеством записей о заказах и бронированиях.

В качестве системы управления базами данных на этапе разработки используется SQLite - встроенное в Python хранилище в одном файле, не требующее отдельного процесса. Для перехода в продуктивную эксплуатацию рассматривается PostgreSQL, поддерживающий полнотекстовый поиск через

типы SearchVector и SearchRank, поиск по триграммам через расширение pg_trgm и инвертированные индексы GIN. Эти средства позволяют построить мощный поиск с учётом морфологии и опечаток без подключения внешних поисковых систем.

Выбор HTMX и Alpine.js вместо полноценного SPA-фреймворка (React, Vue, Angular) обусловлен спецификой задачи: проект небольшой, основная логика концентрируется на сервере, а отдельная команда фронтенд-разработчиков отсутствует. Hypermedia-подход даёт значительное упрощение архитектуры - единая кодовая база, единая модель доступа, отсутствие необходимости в API-слое, сериализаторах, дополнительной сборке JavaScript-приложения и его отдельном развёртывании.

1.3. Принципы функционирования веб-приложения и динамического поиска

Обработка любого HTTP-запроса в Django проходит через несколько последовательных этапов. Сначала запрос поступает в WSGI-сервер (на этапе разработки - встроенный сервер «gunserver», на продуктивном контуре - Gunicorn под управлением Nginx). Django последовательно применяет к запросу middleware - компоненты, отвечающие за CSRF-проверку, аутентификацию, обработку сессий и собственно прикладные задачи. Затем URL-резолвер сопоставляет путь запроса с конфигурацией маршрутов и передаёт управление соответствующему представлению (view). Представление при необходимости обращается к ORM, получает данные и рендерит шаблон. Сгенерированный HTML-ответ проходит через middleware в обратном порядке и отправляется клиенту.

Обработка HTMX-запроса отличается лишь форматом ответа: вместо полной страницы возвращается её фрагмент. Сервер может также установить специальные HTTP-заголовки - «HX-Trigger» (вызывает кастомные события на

стороне клиента, что используется для показа всплывающих уведомлений или очистки корзины) и «HX-Location» (инициирует переход на другую страницу с сохранением состояния клиента). За счёт меньшего размера ответа и отсутствия повторного парсинга всей страницы такой обмен ощущается мгновенным.

Принцип построения агрегатора сводится к идее единой сущности услуги. В разрабатываемом приложении модель Service описывает любую позицию меню - будь то блюдо кухни или напиток с бара. Категоризация осуществляется через справочную модель Category, дополнительные характеристики - через теги (модель Tag). Столы и кальяны выделены в отдельные сущности, так как имеют принципиально иную семантику: стол можно забронировать, а кальян может быть добавлен к брони как опциональная услуга. Такая структура позволяет наращивать каталог без изменения кода: достаточно добавить новые записи через административную панель.

Динамический поиск реализован как последовательность событий, происходящих между клиентом и сервером. Пользователь набирает текст в поле ввода; HTMX через атрибут «hx-trigger» с параметром «input changed delay:300ms, search» дожидается паузы в 300 миллисекунд после последнего нажатия клавиши; затем выполняется GET-запрос на серверное представление с переданным параметром «q». Серверный код проверяет, что запрос пришёл именно от HTMX (по флагу «request.htmx», добавленному пакетом django-htmx), фильтрует объекты модели Service по совпадению с поисковой строкой и рендерит шаблон-фрагмент «_catalog_cards.html». Полученный HTML вставляется в блок с идентификатором «catalog-grid» в режиме «hx-swap=innerHTML», при этом параметр «hx-push-url=false» исключает запись поискового адреса в историю браузера.

Отдельная особенность реализации связана с использованием SQLite на этапе разработки. Эта СУБД некорректно обрабатывает функцию LOWER для строк в кириллице, из-за чего стандартный фильтр «icontains» работает

неустойчиво. Чтобы обеспечить корректный поиск независимо от регистра, фильтрация выполнена на стороне Python: для каждой услуги формируется строка-стог сена (haystack), объединяющая название, описание, наименование категории и теги, после чего проверяется вхождение поискового запроса методом «in». Каталог невелик, и потери производительности от такого подхода незаметны. При переносе на PostgreSQL предполагается заменить этот алгоритм на полнотекстовый поиск с использованием SearchVector и SearchRank, а для устойчивости к опечаткам подключить расширение pg_trgm и метод trigram_similar. Эти структуры ускоряются инвертированным индексом GIN, что обеспечивает время отклика ниже 50 миллисекунд на каталог в десятки тысяч позиций.

Поток данных в полностью развёрнутом приложении выглядит следующим образом: клиентский браузер устанавливает соединение с обратным прокси Nginx; Nginx завершает TLS-соединение и обрабатывает запросы статики самостоятельно, минуя Python; динамические запросы передаются по протоколу WSGI в процесс Gunicorn; Gunicorn запускает несколько процессов Django, каждый из которых обращается к PostgreSQL через пул соединений. Ответы возвращаются в обратном порядке. Такая многоступенчатая схема позволяет выдерживать высокую нагрузку, разделяя ответственность за обработку статики, балансировку соединений и прикладную логику.

Безопасность веб-приложения обеспечивается на нескольких уровнях. Защита от подделки межсайтовых запросов (CSRF) реализована через токены, добавляемые во все формы и проверяемые соответствующим middleware. Для HTMX-запросов токен прокидывается в HTTP-заголовок «X-CSRFToken» через перехватчик события «htmx:configRequest». Защита от межсайтового скриптинга (XSS) основана на автоматическом экранировании переменных в шаблонах Django. Защита от SQL-инъекций гарантируется использованием параметризованных запросов через ORM. Защита от кликджекинга реализуется

HTTP-заголовком «X-Frame-Options: DENY». Пароли пользователей хранятся в виде хешей, сгенерированных алгоритмом PBKDF2 с SHA-256 в качестве хэш-функции, что соответствует современным требованиям к стойкости.

Дополнительный уровень безопасности обеспечивает кастомный middleware «HideAdminFromNonStaffMiddleware», возвращающий HTTP 404 при попытке открыть административную страницу пользователем без признака «is_staff». Это решает не функциональную, а защитную задачу: внешний наблюдатель не получает подтверждения того, что административный интерфейс по адресу «/admin/» действительно существует, что снижает вероятность целенаправленных атак на форму входа.

ГЛАВА 2. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ БАРА «ПРИБОЙ»

2.1. ER-модель базы данных

Проектирование базы данных начинается с выделения основных сущностей предметной области и связей между ними. Для веб-приложения агрегатора услуг бара «Прибой» в результате анализа предметной области выделено пятнадцать сущностей, которые логически распадаются на четыре группы. Первая группа - служебные сущности, связанные с пользователями: «User» (пользователь системы), «EmailVerification» (одноразовый код подтверждения адреса электронной почты при регистрации) и «PasswordReset» (одноразовый код восстановления пароля). Вторая группа - справочные сущности: «Category» (категория позиции меню), «Tag» (тег для поиска и фильтрации), «Promotion» (акция или специальное предложение). Третья группа - каталожные сущности: «Service» (позиция меню или услуга), «Image» (изображение услуги), «Table» (столик бара), «TableImage» (изображение столика), «Hookah» (кальян). Четвёртая группа - транзакционные сущности: «Booking» (бронирование стола), «Order» (предзаказ на вынос), «OrderItem» (позиция предзаказа), «Review» (отзыв о посещении или позиции меню).

Связи между сущностями организованы преимущественно по принципу «один ко многим». Один пользователь может иметь несколько бронирований, несколько заказов, несколько отзывов и несколько кодов подтверждения электронной почты. Одна категория объединяет множество позиций меню; один стол может быть забронирован многократно (в разное время); один кальян может фигурировать в нескольких бронированиях, поскольку поле «hookah» в модели «Booking» допускает «null». Связь «многие ко многим» используется единственный раз - между моделями «Service» и «Tag», что позволяет одной позиции иметь несколько меток и одной метке объединять несколько позиций. ER-диаграмма базы данных приведена на рисунке 1.

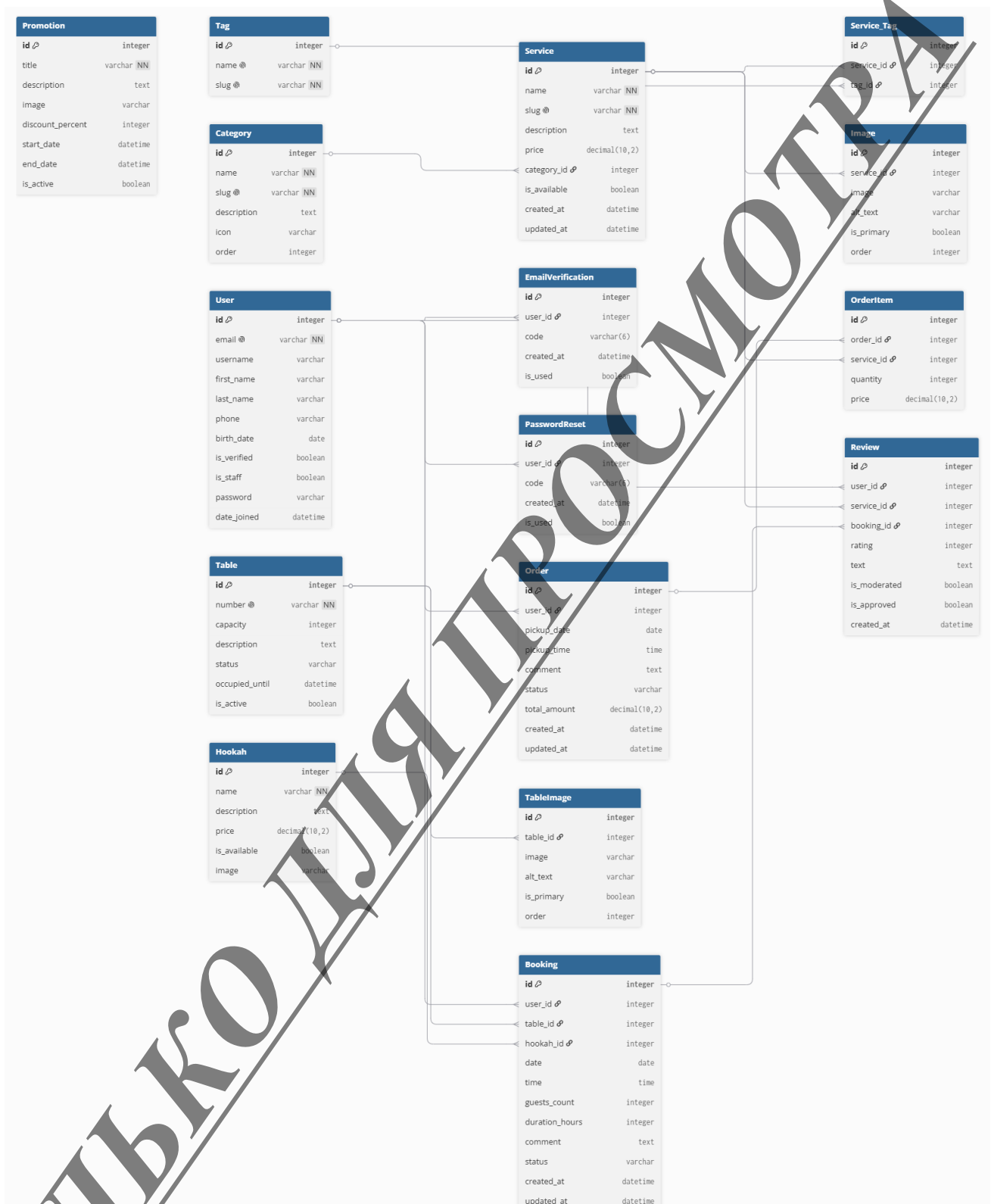


Рисунок 1 - ER-диаграмма базы данных приложения

В проекте сознательно поддерживается третья нормальная форма (3НФ). Каждая таблица обладает первичным ключом - целочисленным полем «id», создаваемым Django по умолчанию. Все неключевые атрибуты функционально зависят только от первичного ключа: например, в таблице «Service» поля «name», «slug», «description», «price», «is_available» и временные метки относятся непосредственно к позиции меню. Транзитивные зависимости исключены: атрибуты категории не дублируются в таблице услуг, а связываются по внешнему ключу «category_id». Аналогично разрешён ассоциативный переход «многие ко многим»: вместо хранения списка тегов в строке таблицы «Service» используется промежуточная таблица «catalog_service_tags», автоматически создаваемая Django ORM при миграции.

Стратегия эволюции схемы базы данных опирается на встроенный механизм миграций Django. Каждое изменение модели - добавление нового поля, переименование, изменение типа, появление новой таблицы - фиксируется в отдельном файле миграции командой «python manage.py makemigrations». Эти файлы хранятся в каталоге «migrations/» каждого приложения и попадают в систему контроля версий вместе с исходным кодом, что обеспечивает воспроизводимость структуры базы на любой среде разработки. Применение миграций к конкретной базе выполняется командой «python manage.py migrate»; служебная таблица «django_migrations» хранит список уже применённых миграций, благодаря чему повторный запуск команды безопасен.

Для ускорения операций поиска и сортировки Django автоматически создаёт индексы на полях с ограничением «unique=True» (например, «slug» у «Category» и «Service», «email» у «User»), на всех внешних ключах, а также на полях, указанных в опции «Meta.ordering». Дополнительная индексация может быть добавлена точно по мере роста нагрузки - например, индекс по полю «status» в таблицах «bookings_booking» и «bookings_order» при увеличении количества записей.

2.2. Создание подсистем (Django-приложений)

Веб-приложение разделено на пять прикладных Django-приложений и один корневой пакет конфигурации. Корневой пакет «priboi» содержит модули «settings.py» (настройки фреймворка), «urls.py» (карта маршрутов верхнего уровня), «wsgi.py» и «asgi.py» (точки входа для WSGI- и ASGI-серверов), а также собственный модуль «middleware.py» с кастомной защитой административной зоны.

Прикладные приложения распределяют ответственность по функциональным областям. Приложение «core» содержит общесистемные представления (главную страницу), management-команды для заполнения базы тестовыми данными и контекстные процессоры, передающие во все шаблоны общие данные - список доступных столов и кальянов для модального окна заказа. Приложение «catalog» отвечает за каталог: модели «Category», «Tag», «Service», «Image», «Table», «TableImage», «Hookah» и «Promotion», а также представления отображения каталога, карточки услуги, страницы стола и поиска в каталоге.

Приложение «bookings» содержит транзакционные сущности - «Booking», «Order», «OrderItem», «Review» - и связанные с ними представления, формы и обработчики HTMX-запросов. Приложение «accounts» инкапсулирует логику работы с пользователями: регистрация, вход и выход, верификация e-mail, восстановление пароля, личный кабинет с историей бронирований и заказов. Приложение «search» реализует глобальный поиск, доступный из модального окна заказа: оно ищет одновременно по позициям меню и по кальянам, возвращая компактную HTML-выборку.

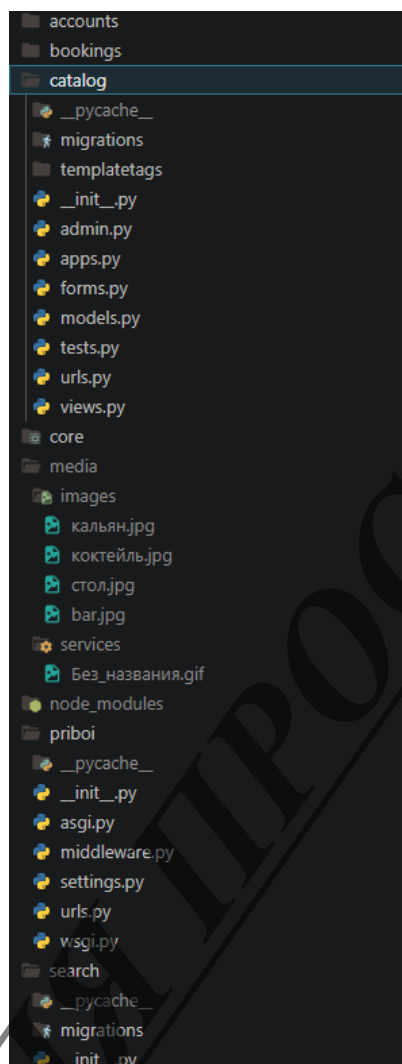


Рисунок 2 - Структура проекта в файловой системе

Подключение приложений в систему выполняется в файле «priboi/settings.py» путём добавления их кодовых имён в список «INSTALLED_APPS». Регистрация маршрутов делается через включение локальных «urls.py» каждого приложения в корневой «priboi/urls.py» с помощью функции «include()», что позволяет каждому приложению самостоятельно отвечать за свой набор адресов.

Список установленных приложений в файле «settings.py» приведён в листинге 1. Порядок имеет значение: пакет «jazzmin» регистрируется первым, чтобы корректно подменить шаблоны Django Admin, а собственные приложения проекта подключаются после служебных модулей фреймворка.

```
INSTALLED_APPS = [  
    'jazzmin',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'django_htmx',  
    'core',  
    'catalog',  
    'bookings',  
    'accounts',  
    'search',  
]
```

Листинг 1 - Установленные приложения

2.3. Справочники и вспомогательные сущности

Справочные сущности служат для классификации основных объектов каталога и для управления контентом. В разработанном приложении эту роль выполняют три модели: «Category», «Tag» и «Promotion». Каждая из них реализует базовые требования к справочнику: уникальный идентификатор, удобочитаемое имя, человекочитаемый строковый идентификатор «slug» для формирования URL.

Модель «Category» имеет, помимо обязательного поля «name», атрибут «order» - целое число, по которому категории сортируются в каталоге. Поле «icon» зарезервировано для будущего использования и может хранить CSS-класс пиктограммы (например, из набора FontAwesome), отображаемой рядом с названием категории во фронтонной части и в навигации Jazzmin. Поле

«description» носит информативный характер и не отображается в публичной выдаче, но удобно для администратора при ведении справочника.

В листинге 2 приведено объявление модели «Category» - наиболее характерного представителя справочной сущности.

```
class Category(models.Model):
```

```
    """Категории услуг"""
```

```
    name = models.CharField('Название', max_length=100)
```

```
    slug = models.SlugField('Slug', unique=True)
```

```
    description = models.TextField('Описание', blank=True)
```

```
    order = models.PositiveIntegerField('Порядок', default=0)
```

```
class Meta:
```

```
    verbose_name = 'Категория'
```

```
    verbose_name_plural = 'Категории'
```

```
    ordering = ['order', 'name']
```

```
def __str__(self):
```

```
    return self.name
```

Листинг 2 - Класс «Category»

Модель «Tag» содержит название и slug. Теги связаны с услугами через ManyToManyField. Одна позиция может иметь несколько тегов («вегетарианское», «сезонное», «новинка»), а один тег - объединять услуги из разных категорий.

Модель «Promotion» описывает акцию: заголовок, описание, изображение, скидку (проценты или фиксированная), даты начала и конца, флаг активности. Активные акции выводятся на главной. Администратор может временно деактивировать акцию без удаления.

Регистрация справочных моделей - через декоратор `@admin.register`. Для `CategoryAdmin` заданы: `list_display` (`name`, `slug`, `order`), `list_editable` (`order` - порядок можно менять прямо в таблице), `search_fields` (поиск по названию), `prepopulated_fields` (`slug` автоматически из названия).

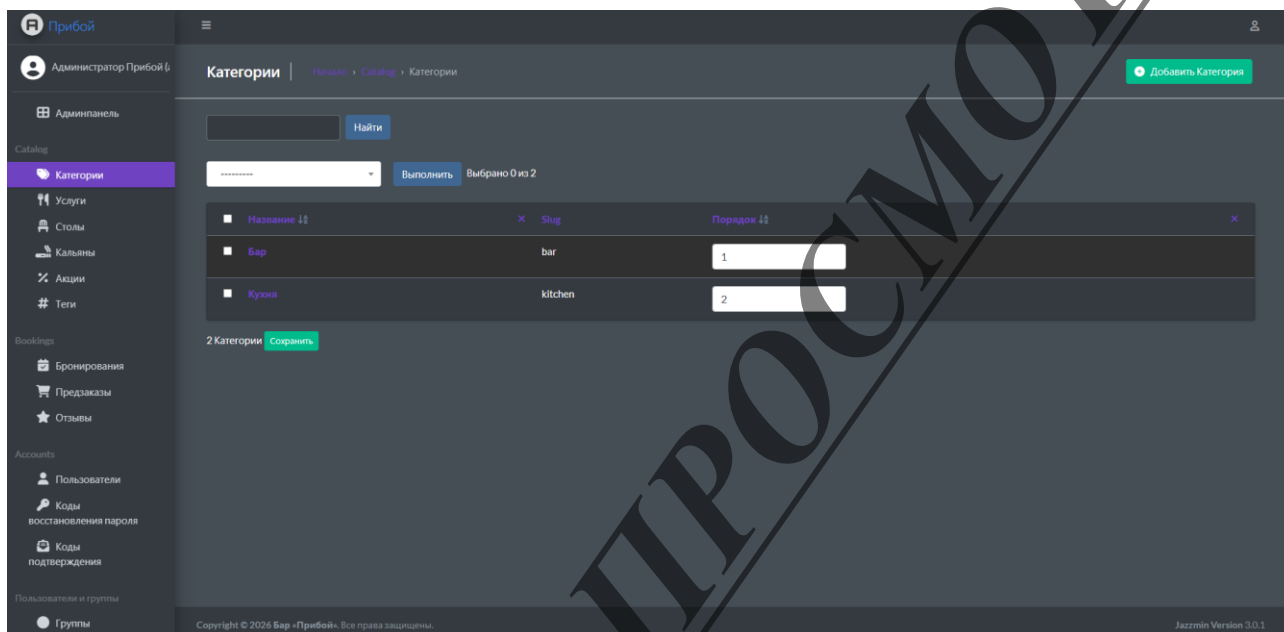


Рисунок 3 - Управление категориями в административной панели

Заполнение справочников начальными данными выполняется management-командой `«create_test_data»`, расположенной в приложении `«core»`. Команда создаёт типовой набор категорий (бар, кухня, кальяны, доставка), несколько десятков тегов, демонстрационные акции и сопоставимое количество позиций меню. Запуск команды выполняется одной строкой: `«python manage.py create_test_data»`, что особенно полезно при развёртывании демонстрационного стенда или при работе нового разработчика с чистой базой.

2.4. Основные сущности заказа и бронирования

Главными сущностями приложения являются позиция меню «Service», столик «Table», кальян «Hookah», бронирование «Booking», предзаказ навынос «Order» с его позициями «OrderItem», а также отзыв «Review». Именно эти модели обеспечивают основной поток действий клиента - выбор позиции, добавление в корзину, оформление заказа и бронирование столика с кальяном.

Листинг 3 демонстрирует объявление модели «Service» - центральной сущности каталога.

```
class Service(models.Model):
    """Услуга/позиция меню"""
    name = models.CharField('Название', max_length=200)
    slug = models.SlugField('Slug', unique=True)
    description = models.TextField('Описание')
    price = models.DecimalField('Цена', max_digits=10,
                               decimal_places=2,
                               validators=[MinValueValidator(0)])
    category = models.ForeignKey(Category,
                                on_delete=models.CASCADE,
                                related_name='services')
    tags = models.ManyToManyField(Tag,
                                related_name='services',
                                blank=True)
    is_available = models.BooleanField('Доступно', default=True)
    created_at = models.DateTimeField('Создано', auto_now_add=True)
    updated_at = models.DateTimeField('Обновлено', auto_now=True)
```

Листинг 3 - Класс Service

Поле «slug» используется для формирования постоянного URL позиции вида «/catalog/service/mojito-classic/». Уникальность slug гарантирует Django через ограничение «unique=True». Поле «price» имеет тип «DecimalField» с двумя знаками после запятой и валидатором «MinValueValidator(0)», запрещающим отрицательные цены. Поле «is_available» позволяет временно скрывать позицию из клиентского каталога без удаления записи и истории заказов с ней. Поля «created_at» и «updated_at» с параметрами «auto_now_add» и «auto_now» соответственно обеспечивают автоматическое заполнение меток времени без участия программиста.

Изображения позиции вынесены в отдельную модель «Image» с обратной связью «related_name='images'» от «Service». Это позволяет на одной странице карточки услуги показывать целую галерею, а в административной панели редактировать список изображений inline-блоком прямо на форме услуги. Поле «is_primary» помечает основное изображение, используемое при отображении карточки в каталоге.

Рисунок 4 - Карточка услуги в административной панели

Модель «Table» описывает столик бара. Поле «number» хранит отображаемый клиенту идентификатор стола и сделано уникальным; «capacity» - допустимое количество гостей; «status» принимает одно из трёх значений: «free» (свободен), «occupied» (занят) или «reserved» (забронирован). Дополнительное поле «occupied_until» хранит время, до которого стол занят: это позволяет клиентскому интерфейсу и логике бронирования корректно учитывать пересечения. Флаг «is_active» нужен для временного исключения стола из оборота, например, при ремонте мебели или перепланировке зала.

Модель «Hookah» описывает кальян как самостоятельную услугу. Атрибуты модели стандартны: название/вкус, описание, цена, флаг доступности, опциональное изображение. Кальян не имеет статусной логики, так как одновременно может быть подан несколько кальянов разных вкусов; ограничения по фактическому количеству оборудования в зале регулируются персоналом, а не системой.

Главной транзакционной сущностью клиентского сценария является «Booking» - бронирование столика. Запись хранит связь с пользователем, столом, при необходимости - с кальяном, дату и время начала брони, её длительность в часах, число гостей, произвольный комментарий и статус. Перечисление статусов «STATUS_CHOICES» содержит четыре значения: «pending» (ожидает подтверждения), «confirmed» (подтверждено), «cancelled» (отменено), «completed» (завершено).

Объявление модели «Booking» представлено в листинге 4.

```
class Booking(models.Model):
    """Бронирование столика"""

    STATUS_CHOICES = [
        ('pending', 'Ожидает подтверждения'),
        ('confirmed', 'Подтверждено'),
        ('cancelled', 'Отменено'),
```

```

        ('completed', 'Завершено'),
    ]
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
                             on_delete=models.CASCADE,
                             related_name='bookings')
    table = models.ForeignKey(Table,
                              on_delete=models.CASCADE,
                              related_name='bookings')
    date = models.DateField('Дата')
    time = models.TimeField('Время')
    guests_count = models.PositiveIntegerField('Гостей')
    duration_hours = models.PositiveIntegerField('Длительность',
                                                default=2)
    hookah = models.ForeignKey(Hookah,
                               on_delete=models.SET_NULL,
                               null=True, blank=True,
                               related_name='bookings')
    status = models.CharField('Статус', max_length=20,
                              choices=STATUS_CHOICES,
                              default='pending')

```

Листинг 4 - Класс «Booking»

Защита от двойного бронирования реализована на уровне представления и модели «Table». При успешном создании брони статус соответствующего стола меняется на «reserved», а в поле «occupied_until» записывается рассчитанное время окончания бронирования. Форма создания бронирования отклоняет запросы, в которых указанный временной интервал пересекается с уже существующей бронью этого же стола. Все изменения совершаются внутри одного транзакционного блока «transaction.atomic()», что гарантирует целостность данных даже при одновременных обращениях нескольких пользователей.

Список бронирований в административной панели поддерживает массовые действия (см. рисунок 5). Метод «approve_bookings» переводит выделенные записи в статус «confirmed», метод «cancel_bookings» - в статус «cancelled». Эти действия зарегистрированы в атрибуте «actions» класса «BookingAdmin» и появляются в выпадающем списке над таблицей. Иерархия дат «date_hierarchy = 'date'» добавляет в верхнюю часть страницы фильтр по годам, месяцам и дням, что упрощает администратору навигацию по большому объёму бронирований.

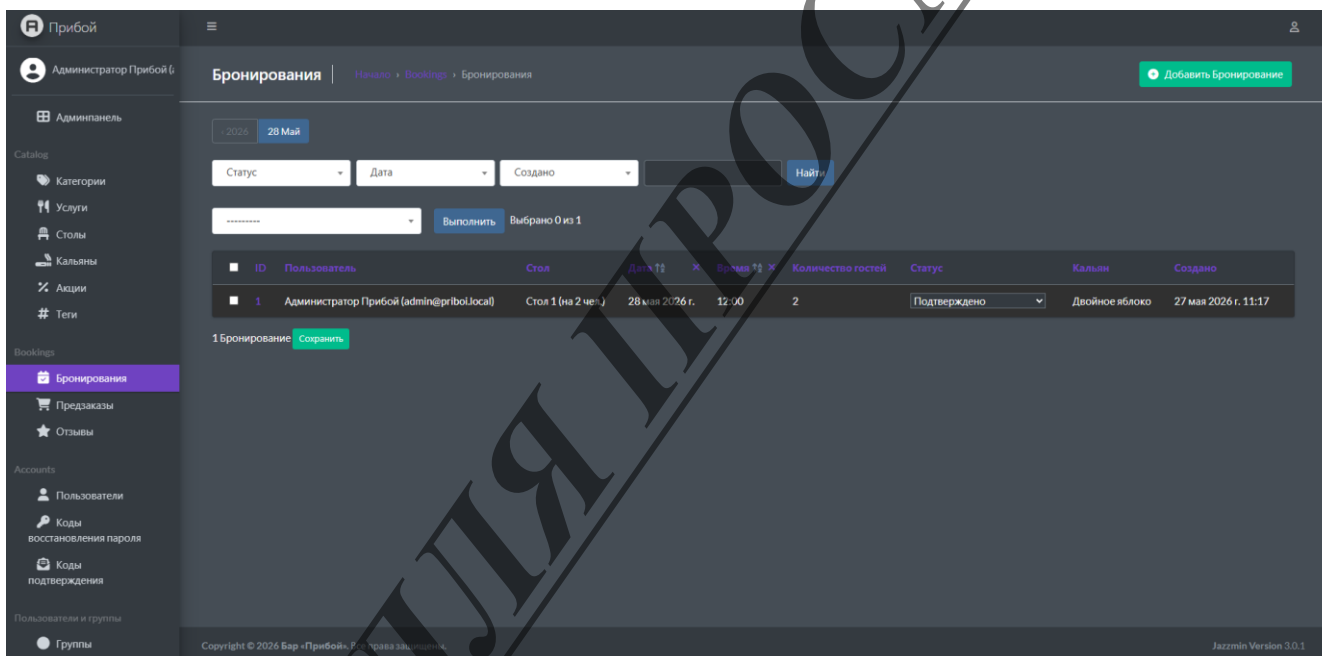


Рисунок 5 - Список бронирований с массовыми действиями

Параллельно с бронированием реализован сценарий предзаказа навынос - модель «Order». Заказ содержит ссылку на пользователя, дату «pickup_date» и время «pickup_time» получения, комментарий, статус, общую сумму «total_amount» и временные метки создания и обновления. Перечисление статусов заказа более развёрнуто, чем у бронирования: «pending», «confirmed», «preparing», «ready», «completed», «cancelled». Дополнительные статусы «preparing» и «ready» отражают внутреннюю кухонную логистику и позволяют персоналу видеть, на каком этапе подготовки находится конкретный заказ.

Метод «calculate_total» суммирует подытоги всех позиций заказа («OrderItem») и сохраняет результат в поле «total_amount». Метод вызывается после создания всех позиций заказа в рамках обработки HTTP-запроса в представлении «create_order». Хранение общей суммы непосредственно в таблице заказа, а не вычисление её на лету, оправдано тем, что цены позиций со временем могут изменяться, а историческое значение заказа должно оставаться неизменным.

В листинге 5 показан фрагмент модели «OrderItem», в котором переопределён метод «save».

```
class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
                              related_name='items')
    service = models.ForeignKey('catalog.Service',
                               on_delete=models.CASCADE)
    quantity = models.PositiveIntegerField('Количество')
    price = models.DecimalField('Цена', max_digits=10,
                                decimal_places=2)

    @property
    def subtotal(self):
        return self.price * self.quantity

    def save(self, *args, **kwargs):
        if not self.price:
            self.price = self.service.price
        super().save(*args, **kwargs)
```

Листинг 5 - Класс «OrderItem»

Каждая позиция заказа («OrderItem») связана с конкретной услугой через внешний ключ, а также имеет собственное поле цены, независимое от текущей цены услуги. Эта избыточность намеренна: она обеспечивает «бухгалтерскую» неизменность исторических данных. Свойство «subtotal» возвращает произведение цены на количество и используется как при пересчёте суммы заказа, так и при отображении строк в интерфейсе.

The screenshot shows the 'Прибой' application interface. On the left is a sidebar with navigation links: Админпанель, Каталог (Категории, Услуги, Столы, Калькуляторы, Акции, Теги), Bookings (Бронирования, **Предзаказы**, Отзывы), Accounts (Пользователи, Коды восстановления пароля, Коды подтверждения), and Пользователи и группы (Группы). The main area displays an order card for 'Администратор Прибой (admin@priboi.local)'.

The order card has two tabs: 'Основная информация' (selected) and 'Позиции заказа'. The 'Основная информация' tab contains the following fields:

- Пользователь:** Администратор Прибой (admin@priboi.local)
- Дата получения:** 27.05.2026 (Today)
- Время получения:** 12:00:00 (Today)
- Комментарий:** (Empty text area)
- Статус:** Ожидает подтвержд...
- Создано:** 27 мая 2026 г. 11:17
- Обновлено:** 27 мая 2026 г. 11:17
- Общая сумма:** 1,00

On the right side of the card, there are buttons: Удалить, Сохранить и добавить другой объект, Сохранить и продолжить редактирование, and История.

Рисунок 6 – Карточка заказа с inline-позициями

Завершает группу транзакционных сущностей модель «Review». Каждый отзыв связан с пользователем-автором и опционально с оцениваемой позицией меню или с конкретным бронированием. Возможность связать отзыв с бронированием позволяет в перспективе агрегировать оценки качества обслуживания, отдельные от оценок самих блюд. Поля «rating» (целое число от 1 до 5), «text», флаги «is_moderated» и «is_approved» поддерживают двухэтапный процесс модерации: администратор сначала просматривает отзыв, а затем принимает решение о его публикации. Неодобренный отзыв виден только своему автору в личном кабинете, что снижает мотивацию повторного создания негативных записей.

2.5. Реализация системы динамического поиска

Динамический поиск является одной из ключевых функций каталога. От его скорости и качества зависит, насколько комфортно пользователю находить нужные позиции в меню. В разработанном приложении реализованы три точки входа в поисковую функциональность. Первая - поле поиска на странице каталога, обновляющее сетку карточек прямо на экране. Вторая - поле поиска внутри модального окна заказа, помогающее пользователю быстро добавить позицию в корзину. Третья - общая поисковая страница, агрегирующая результаты по разным типам сущностей.

Клиентская часть поиска описана атрибутами HTMX прямо на теге `<input type="search">`. Запрос уходит при изменении поля с задержкой 300 мс, а также при нажатии Enter. Результат подставляется в блок `#catalog-grid` через `innerHTML`. Запись в историю браузера отключена (`hx-push-url="false"`), чтобы не засорять кнопку «Назад». Индикатор загрузки - элемент `#search-indicator` с классом `.spinner`.

Листинг 6 показывает разметку поискового поля в каталоге со всеми HTMX-атрибутами.

```
<input type="search"
      name="q"
      class="input input-icon-left"
      hx-get="{% url 'catalog:search' %}"
      hx-trigger="input changed delay:300ms, search"
      hx-target="#catalog-grid"
      hx-swap="innerHTML"
      hx-push-url="false"
      hx-indicator="#search-indicator">
```

Листинг 6 - Разметка

Серверный поиск реализован в представлении search приложения catalog. Из-за SQLite, которая некорректно обрабатывает LOWER для кириллицы, стандартный фильтр name_icontains работает ненадёжно. Поэтому фильтрация вынесена на сторону Python.

Для каждой услуги формируется строка-«стог сена» из названия, описания, категории и тегов. Запрос сравнивается с ней через оператор in. Каталог небольшой (десятки позиций), накладные расходы незаметны, а корректность работы с кириллицей гарантирована.

Фрагмент серверного кода с реализацией Python-фильтра по haystack приведён в листинге 7.

```
def _service_haystack(service):
    parts = [service.name or "", service.description or ""]
    if service.category_id:
        parts.append(service.category.name or "")
    parts.extend(t.name for t in service.tags.all())
    return ' '.join(parts).lower()

def _services_qs(query):
    qs = Service.objects.select_related(
        'category'
    ).prefetch_related(
        'images', 'tags'
    ).filter(is_available=True)
    query = (query or "").strip().lower()
    if not query:
        return list(qs)
    return [s for s in qs if query in _service_haystack(s)]
```

Листинге 7 - haystack

Для оптимизации выборки используются методы «select_related» (включает связанную модель «Category» в один SQL-запрос через JOIN) и «prefetch_related» (выполняет отдельный запрос для обратных связей «images» и «tags», но с одним обращением к базе на весь набор). Эта пара методов устраняет проблему «N+1 запроса», характерную для ORM при обходе связанных объектов в цикле.

При переходе на PostgreSQL планируется заменить Python-фильтр на встроенный полнотекстовый поиск. Для каждой позиции будет храниться предвычисленный «SearchVector», в котором значимым полям присвоены весовые коэффициенты: названию - самый высокий вес «A», описанию - средний «B», категории - «C», тегам - «D». Серверная функция «SearchRank» позволит сортировать выдачу по релевантности. Для устойчивости к опечаткам предполагается подключить расширение «pg_trgm» и использовать оператор «trigram_similar», а инвертированный индекс GIN на поле «search_vector» обеспечит время отклика ниже 50 миллисекунд даже при каталоге в десятки тысяч позиций.

Фильтрация по категориям реализована полностью на стороне клиента - без сетевого запроса. Кнопки вкладок выше сетки карточек переключают переменную «tab» в локальном состоянии Alpine.js, а каждая карточка реагирует на это значение атрибутом «x-show», скрывая или показывая себя без перерисовки страницы. Такой подход возможен потому, что весь каталог отображается одной HTMX-выборкой, и дополнительный сетевой запрос для фильтрации не требуется.

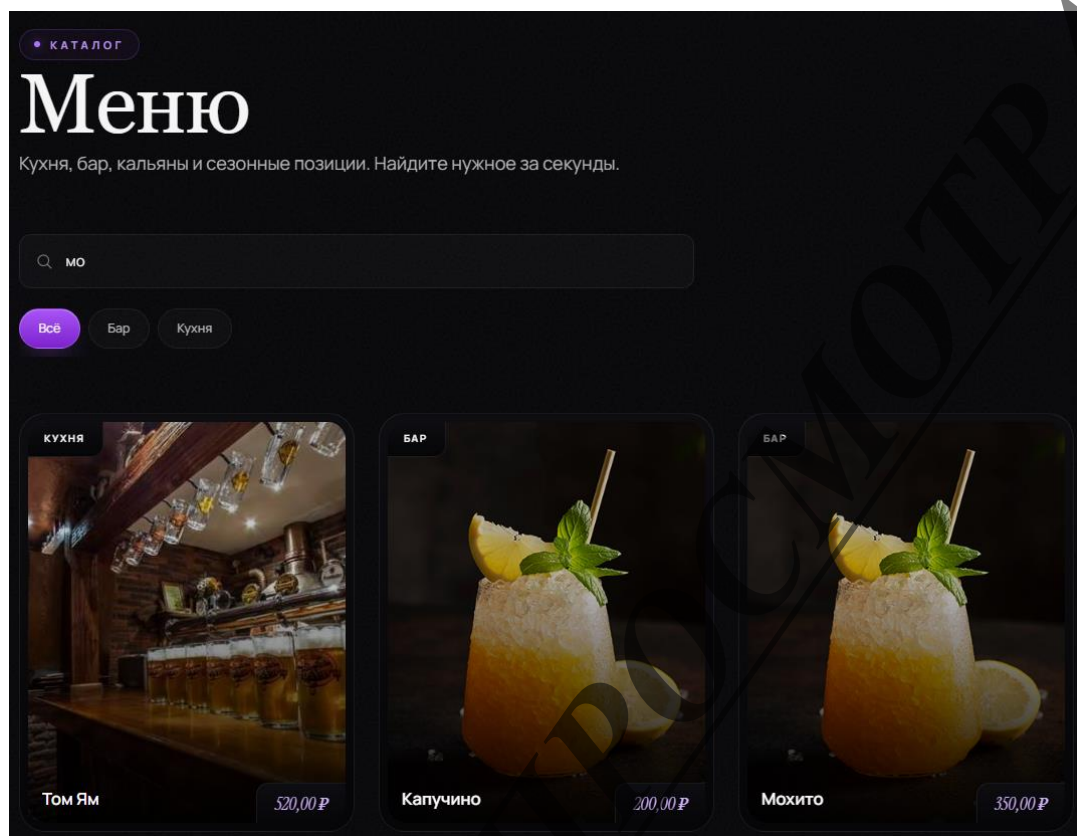


Рисунок 7 - Динамический поиск в каталоге

2.6. Клиентский интерфейс оформления заказа и бронирования

Клиентский интерфейс построен на трёх библиотеках: HTMX - асинхронное взаимодействие с сервером без перезагрузки страницы, Alpine.js - локальная реактивность и глобальное состояние, Tailwind CSS - визуальная система. Базовый шаблон «base.html» подключает скомпилированный CSS, клиентский JS, а также плагины Flatpickr (выбор даты/времени) и Tom-Select (улучшенные выпадающие списки).

Оформление: тёмная палитра с фиолетовыми акцентами (атмосфера бара). Шрифты: Instrument Serif для крупных заголовков, Manrope для основного текста. Карточки услуг выполнены в overlay-стиле: изображение занимает почти всю карточку, а название и цена вынесены в небольшие плашки, врезанные в углы. Общий вид главной страницы - на рисунке 8, страница каталога - на рисунке 9.

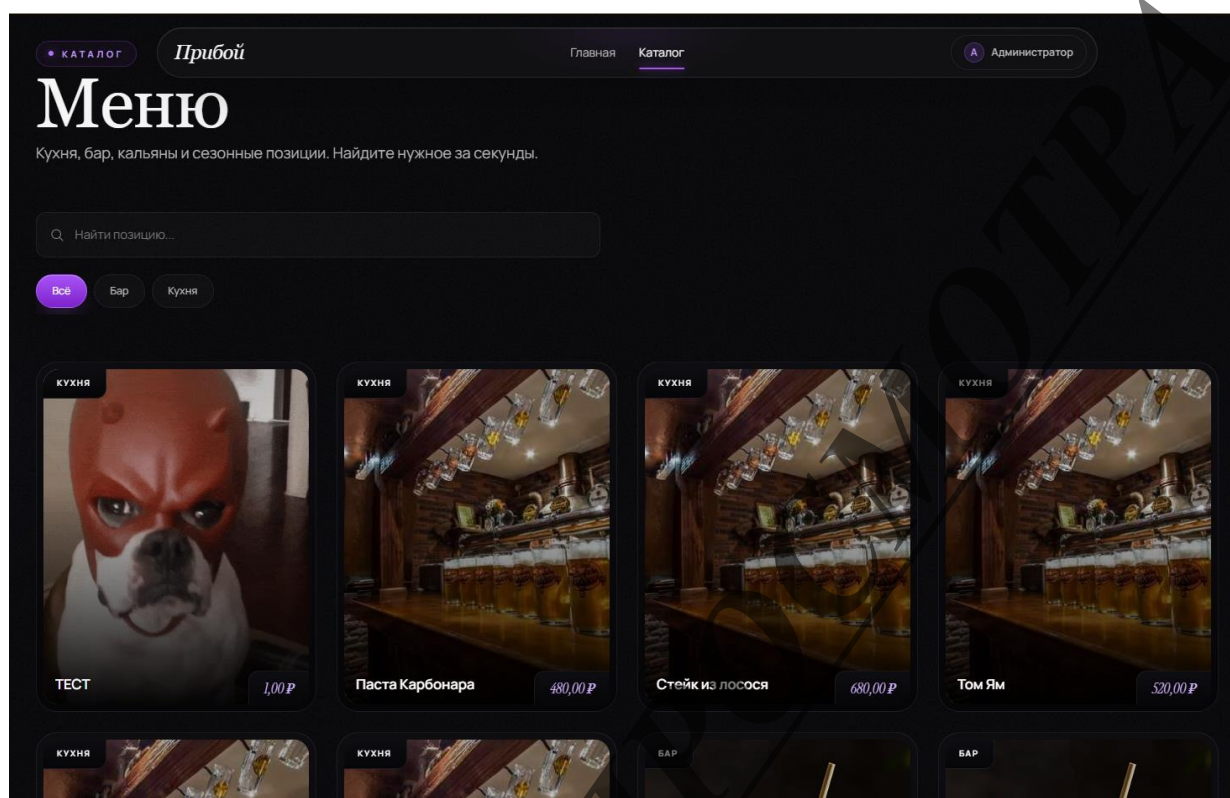


Рисунок 9 - Страница каталога с карточками услуг

Центральным элементом клиентского интерфейса является модальное окно заказа, объединяющее две вкладки - корзину с предзаказом на вынос и бронирование столика. Окно открывается из любого места сайта по нажатию плавающей круглой кнопки FAB или по программному вызову «`$store.modal.show()`». Открытие сопровождается анимацией появления, реализованной через директивы «`x-transition`» Alpine.js с использованием физических кривых сглаживания (cubic-bezier).

Состояние корзины хранится в глобальном хранилище «`Alpine.store('cart')`», объявленном в файле «`app.js`». Хранилище экспортирует методы «`add`», «`remove`», «`adjust`», «`clear`» для управления составом корзины, геттеры «`total`» и «`count`» для агрегатных значений, а также метод «`serialize`», возвращающий компактное JSON-представление корзины для отправки на сервер.

Объявление хранилища корзины в файле «app.js» приведено в листинге 8.

```
Alpine.store('cart', {
  items: [],
  pickupDate: "",
  pickupTime: "",
  add(service) {
    const existing = this.items.find(
      i => i.id === service.id);
    if (existing) {
      existing.quantity += 1;
    } else {
      this.items.push({
        id: service.id,
        name: service.name,
        price: Number(service.price),
        quantity: 1,
      });
    }
  },
  get total() {
    return this.items.reduce(
      (sum, i) => sum + i.price * i.quantity, 0);
  },
});
```

Листинг 8 - Хранилище корзины

Поток оформления заказа выглядит следующим образом. Клиент добавляет позицию в корзину прямо в каталоге или через поиск внутри модального окна. Счётчик позиций на бейдже плавающей кнопки обновляется автоматически благодаря реактивности Alpine.js. При нажатии кнопки «Оформить заказ» внутри модального окна выполняется HTMX-запрос «POST»

на представление «bookings.create_order»; корзина отправляется в виде JSON в параметре «cart» через атрибут «hx-vals», вычисляемый JavaScript-кодом «js:{cart: Alpine.store('cart').serialize()})».

Серверное представление в одной атомарной транзакции создаёт запись «Order», для каждой позиции создаёт «OrderItem» и вызывает метод «calculate_total» для пересчёта общей суммы. В случае успеха клиенту возвращается ответ с двумя специальными HTTP-заголовками: «HX-Trigger» с массивом кастомных событий (всплывающее уведомление, очистка корзины, закрытие модального окна) и «HX-Location» с адресом личного кабинета.

Внешний вид модального окна на вкладке корзины представлен на рисунке 10, на вкладке бронирования - на рисунке 11.

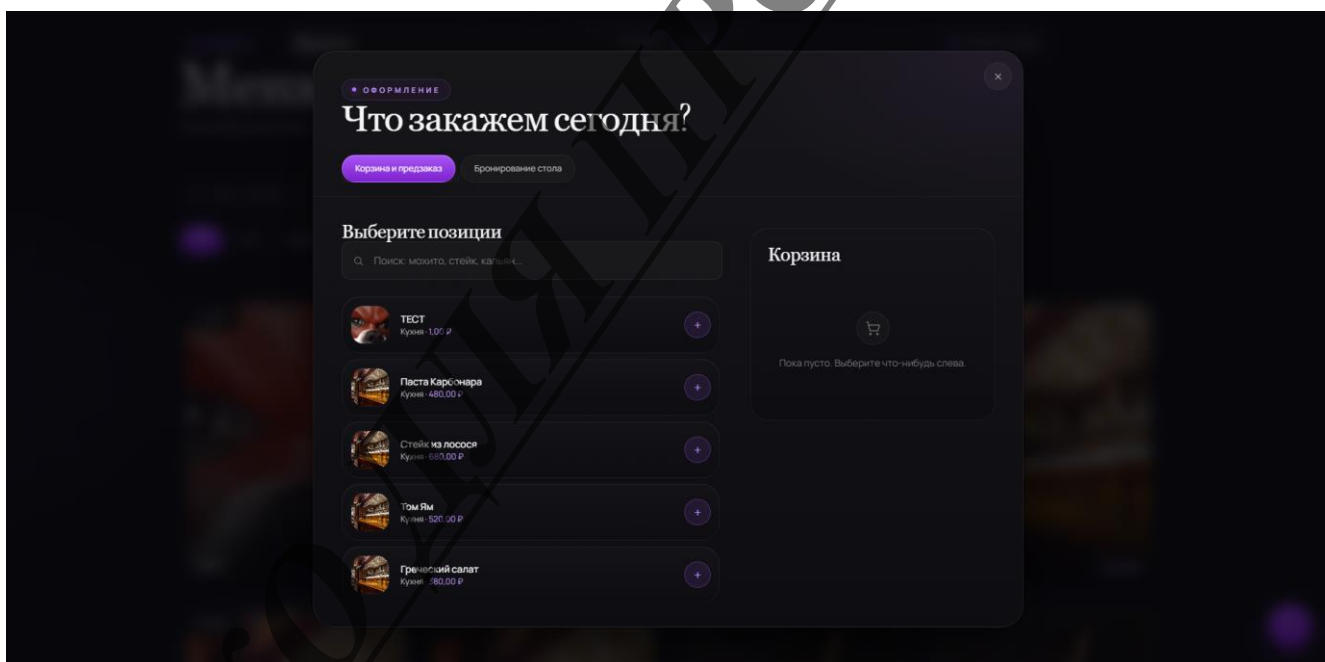


Рисунок 10 - Модальное окно: вкладка корзины и предзаказа

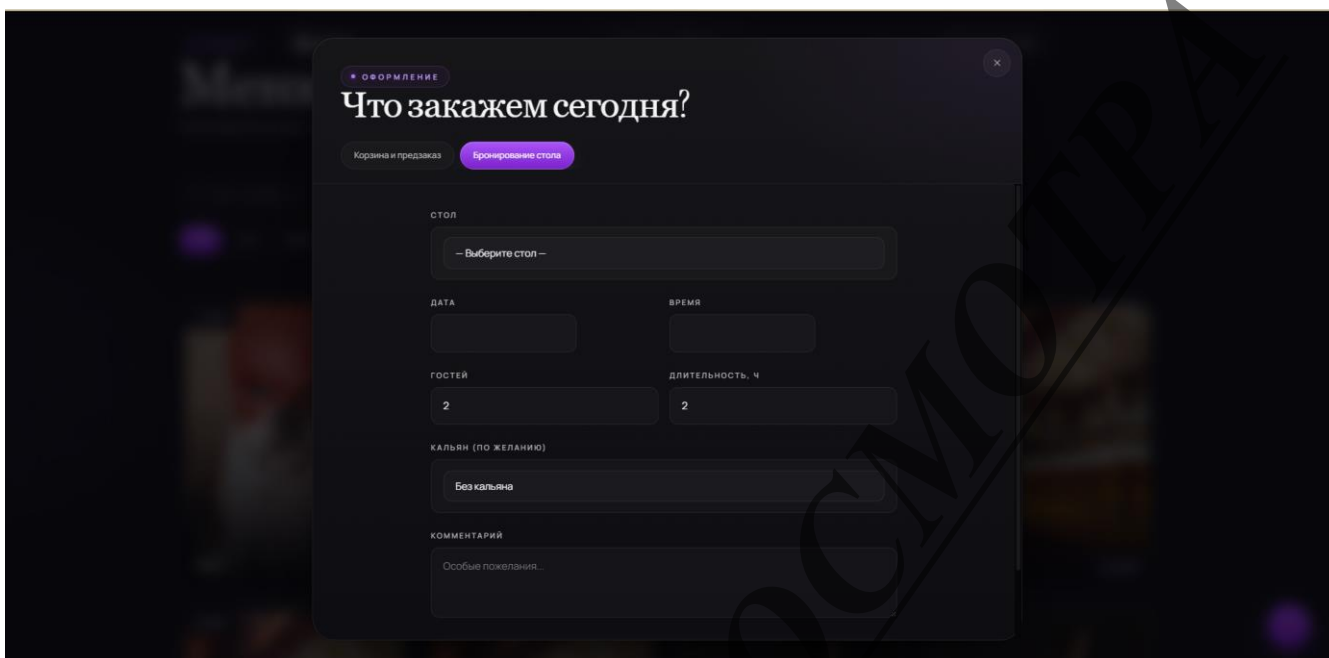


Рисунок 11 - Модальное окно: вкладка бронирования столика

Вкладка бронирования столика устроена как обычная HTML-форма с дополнительной реактивностью на стороне клиента, обеспеченной компонентом «bookingForm» (объявление «Alpine.data»). Компонент хранит реактивные значения количества гостей и длительности брони, а также вычисляет минимальную доступную дату (сегодняшнюю). Дополнительная логика обрабатывает случай, когда модальное окно открывается с уже выбранным столиком - например, нажатием кнопки «Забронировать» на странице конкретного стола: идентификатор стола передаётся через свойство «\$store.modal.prefill» и автоматически проставляется в селекте после открытия окна.

Поля даты и времени украшены плагином Flatpickr с русской локализацией, что обеспечивает единый внешний вид календаря на всех браузерах. Поле выбора стола использует Tom-Select для улучшения отображения и поиска внутри длинного списка. Оба плагина инициализируются после первой загрузки страницы и переинициализируются после каждой

HTMX-замены DOM, что гарантирует их работоспособность в SPA-режиме «hx-boost».

Личный кабинет пользователя устроен как одностраничное приложение в пределах одной страницы: переключение между вкладками «Брони», «Заказы» и «Профиль» выполняется через локальное состояние Alpine.js без обращения к серверу. Вкладка «Брони» показывает активные бронирования и историю; вкладка «Заказы» - активные и завершённые предзаказы на вынос; вкладка «Профиль» позволяет отредактировать имя, фамилию и телефон, а также сменить пароль (см. рисунок 2.12).

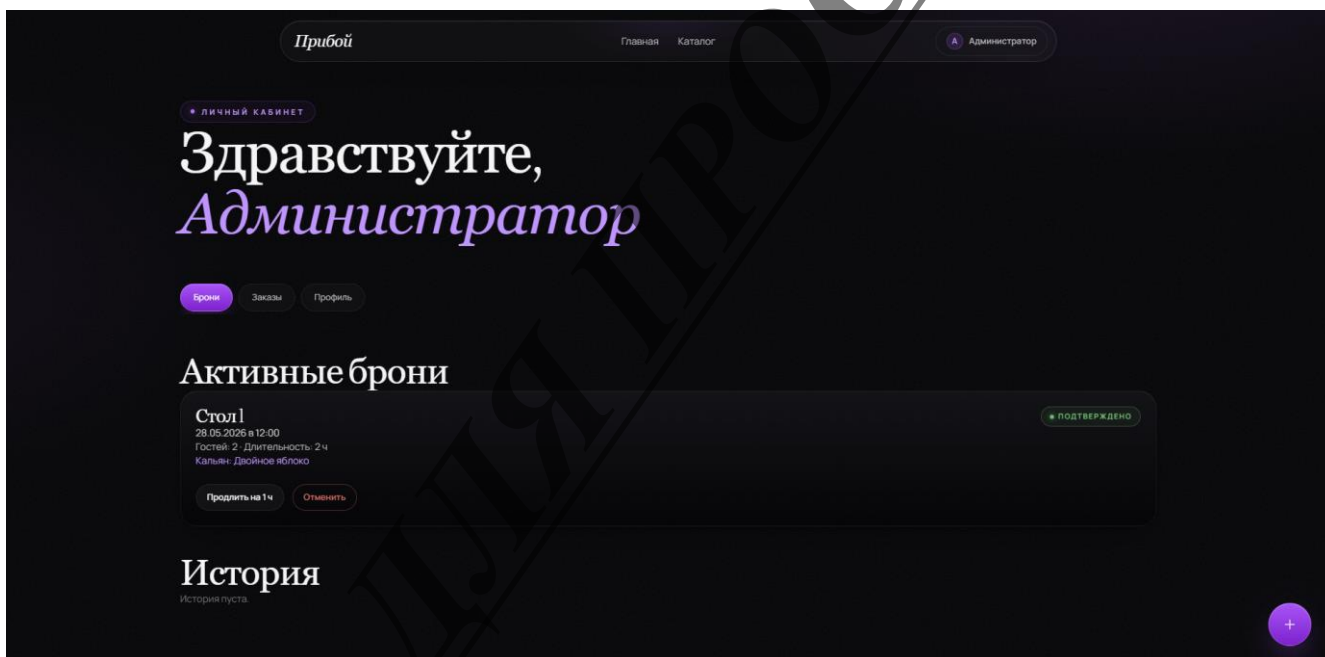


Рисунок 12 - Личный кабинет пользователя

2.7. Административная панель и отчёты (Jazzmin)

Административная панель является ключевым инструментом для персонала бара. От её удобства напрямую зависит скорость обработки заказов и бронирований, поэтому при разработке этой части проекту уделено повышенное внимание. Базовым средством выступает встроенная панель Django Admin, однако её стандартный внешний вид заменён современной темой

Jazzmin на базе Bootstrap 4 с тёмным режимом «darkly» и фиолетовым акцентом.

Подключение Jazzmin выполняется добавлением пакета первым в список «INSTALLED_APPS» - это обязательное условие, поскольку Jazzmin переопределяет шаблоны Django Admin. Настройки темы разделены на два словаря в файле «settings.py»: «JAZZMIN_SETTINGS» хранит брендинг и иконки моделей, «JAZZMIN_UI_TWEAKS» - параметры внешнего вида (цветовая схема, фиксированные панели, отступы).

Базовые настройки темы Jazzmin приведены в листинге 9.

```
JAZZMIN_SETTINGS = {  
    'site_title': 'Прибой - администрирование',  
    'site_header': 'Прибой',  
    'site_brand': 'Прибой',  
    'welcome_sign': 'Панель управления баром «Прибой»',  
    'show_sidebar': True,  
    'navigation_expanded': True,  
    'icons': {  
        'catalog.service': 'fas fa-utensils',  
        'catalog.table': 'fas fa-chair',  
        'catalog.hookah': 'fas fa-smoking',  
        'bookings.booking': 'fas fa-calendar-check',  
        'bookings.order': 'fas fa-shopping-cart',  
    },  
    'changeform_format': 'horizontal_tabs',  
}
```

Листинг 9 - Базовая настройка темы

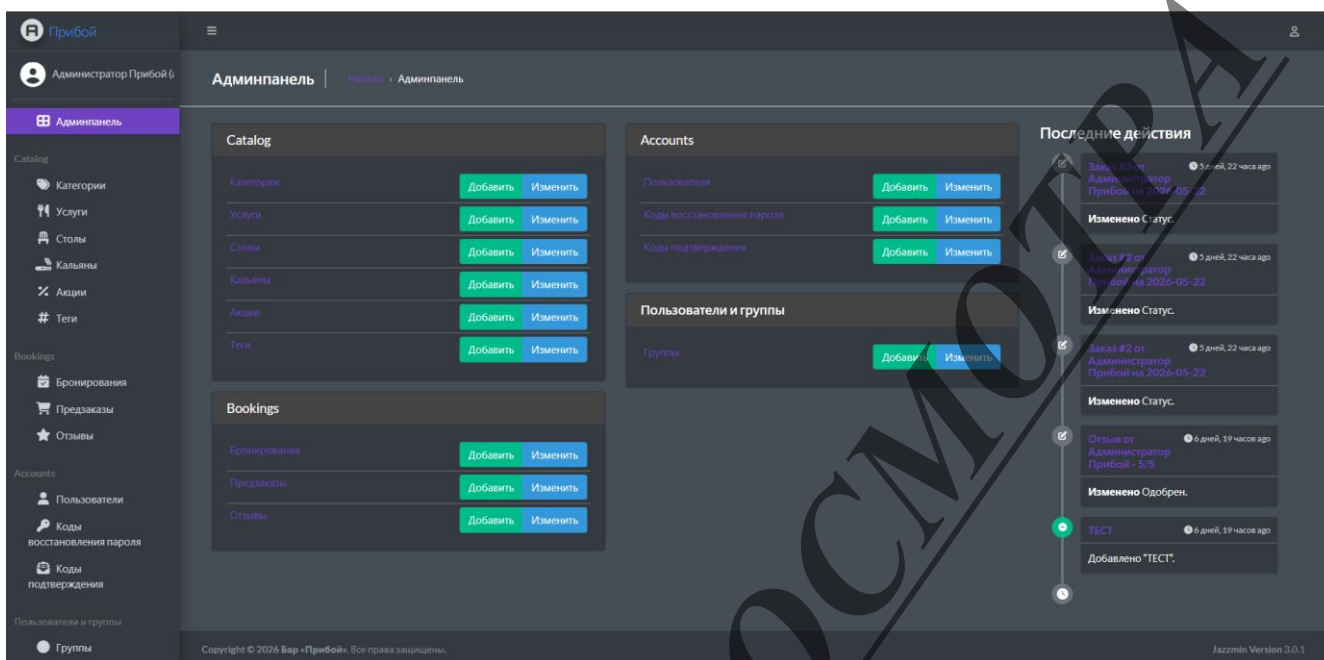


Рисунок 13 - Главная страница административной панели Jazzmin

Каждая модель проекта зарегистрирована в админ-панели через декоратор «`@admin.register`» с классом-конфигурацией, наследующимся от «`admin.ModelAdmin`». В этом классе перечисляются атрибуты, управляющие внешним видом и поведением страницы модели: «`list_display`» (колонки в таблице списка), «`list_filter`» (блок фильтров справа), «`search_fields`» (поле поиска вверху), «`date_hierarchy`» (быстрая навигация по датам), «`list_editable`» (редактирование значений прямо в списке без открытия формы), «`inlines`» (вложенные формы связанных объектов), «`actions`» (массовые операции над выделенными записями).

Для модели «`Service`» определён inline-блок «`ImageInline`», позволяющий администратору загружать несколько изображений позиции прямо со страницы редактирования услуги. Поле «`list_filter`» содержит фильтры по категории и доступности; поле «`date_hierarchy`» добавляет навигатор по датам создания. Поле «`filter_horizontal`» для тегов превращает обычный множественный выбор в удобный двухпанельный виджет с поиском.

Особое значение имеют массовые действия для модели «Booking». Два метода «approve_bookings» и «cancel_bookings», зарегистрированные в атрибуте «actions», позволяют администратору одной операцией перевести несколько броней в статус «confirmed» или «cancelled». Это необходимо в часы пик, когда поступает большое количество новых заявок и обработка каждой в отдельности заняла бы слишком много времени.

Реализация одного из массовых действий показана в листинге 10.

```
@admin.register(Booking)
class BookingAdmin(admin.ModelAdmin):
    list_display = ['id', 'user', 'table', 'date',
                   'time', 'status', 'hookah']
    list_filter = ['status', 'date', 'created_at']
    list_editable = ['status']
    actions = ['approve_bookings', 'cancel_bookings']

    def approve_bookings(self, request, queryset):
        queryset.update(status='confirmed')
        self.message_user(
            request,
            f'Одобрено: {queryset.count()}')
    approve_bookings.short_description = (
        'Одобрить выбранные бронирования')
```

Листинг 10 - Класс «BookingAdmin»

Аналогично организована модель «Order». Атрибут «inline» связывает форму заказа с «OrderItemInline» - позиции заказа редактируются прямо на той же странице. Поле «total_amount» помечено как «readonly_fields», поскольку оно вычисляется автоматически методом «calculate_total» при создании заказа и не должно меняться вручную. Массовые действия «confirm_orders» и «mark_ready» переводят группу заказов в статусы «confirmed» и «ready»

соответственно. Внешний вид страницы списка заказов и страницы редактирования отдельного заказа показаны на рисунках 14 и 15.

Управление отзывами реализовано в «ReviewAdmin». Поле «is_approved» вынесено в «list_editable», что позволяет одобрить отзыв одним кликом без открытия формы. Массовые действия «approve_reviews» и «reject_reviews» дают возможность массовой модерации, причём «reject_reviews» одновременно ставит флаги «is_moderated=True» и «is_approved=False» - это означает, что отзыв рассмотрен и не прошёл модерацию.

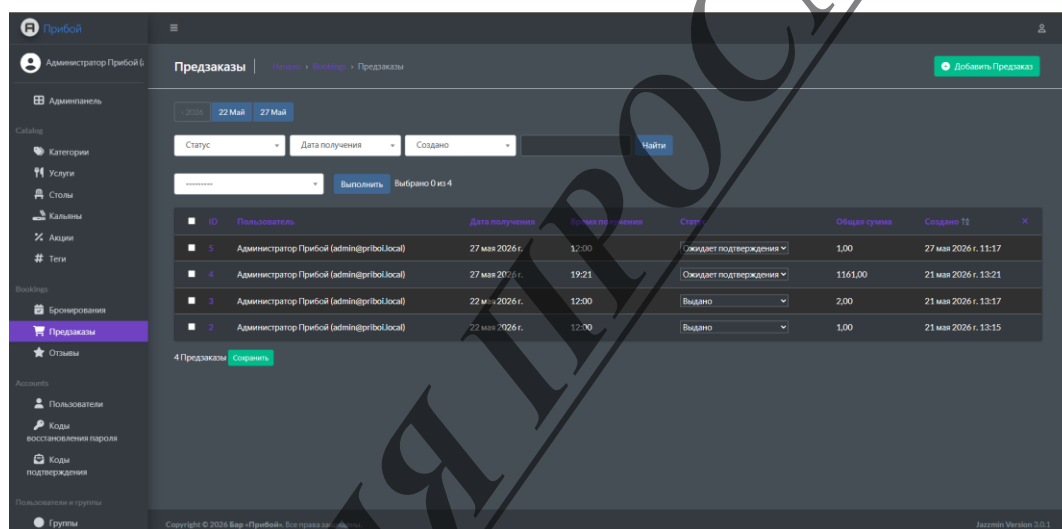


Рисунок 14 - Список заказов в административной панели

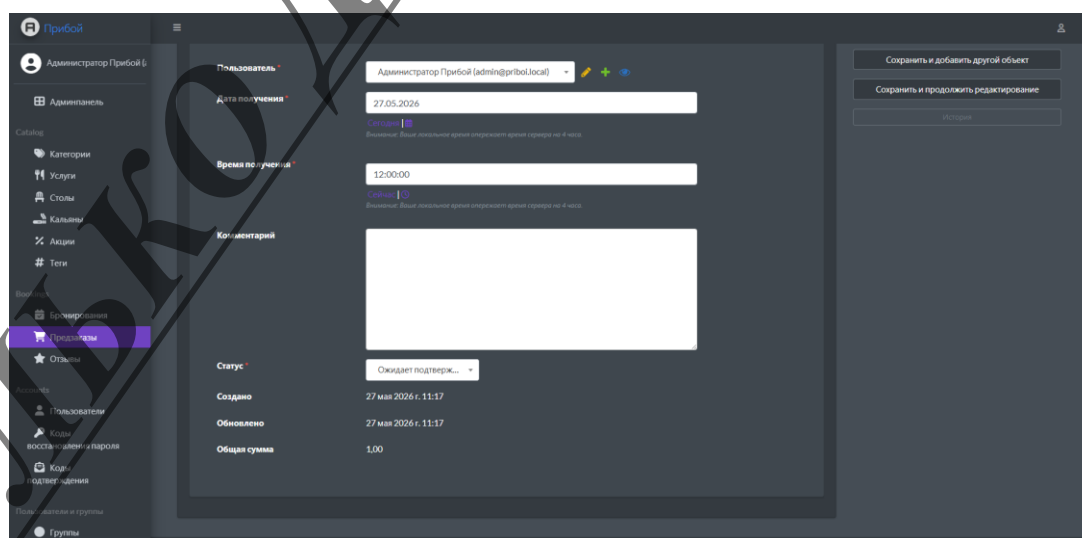


Рисунок 15 - Редактирование заказа с inline-позициями

Базовые отчёты реализованы средствами самой Django Admin. Иерархия дат «date_hierarchy» по полю «date» в «BookingAdmin» позволяет администратору одним кликом увидеть бронирования за конкретный день, месяц или год. Сортировка списка заказов по столбцу «total_amount» в порядке убывания фактически отвечает на вопрос «какие заказы принесли наибольшую выручку». Подсчёт количества заказов в определённом статусе доступен через фильтры списка с числовым индикатором.

Развитие отчётной части предполагается в направлении подключения специализированного пакета - например, «django-import-export» - для выгрузки выборок в формат CSV или Excel, а также добавления отдельной admin-страницы со сводными графиками на основе библиотеки Chart.js. Эти задачи остаются за рамками текущей работы как направление дальнейшего развития.

2.8. Пользователи и система аутентификации

Система аутентификации построена на Django с отличием: вместо имени пользователя основным идентификатором является email. Создана кастомная модель User (наследует AbstractUser) с полями: phone (с валидатором), birth_date, email (unique=True) и булевым флагом is_verified.

Регистрация - двухэтапная:

- Пользователь заполняет форму (email, имя, фамилия, телефон, пароль).
- На email приходит 6-значный код, который сохраняется в модели EmailVerification (related_name='verifications'). После ввода кода флаг is_verified становится True, запись помечается is_used=True.

Восстановление пароля устроено аналогично: пользователь вводит email, получает код, затем вводит код и новый пароль. Коды хранятся в модели PasswordReset.

Защита маршрутов: декоратор `@login_required` для оформления заказа, бронирования, отмены заказа, добавления отзыва. Анонимный пользователь перенаправляется на страницу входа с возвратом к исходному маршруту.

Дополнительная защита: кастомный middleware `HideAdminFromNonStaffMiddleware` перехватывает запросы к `/admin/` и для пользователей без `is_staff` (включая анонимов) возвращает HTTP-404, скрывая факт существования админ-панели.

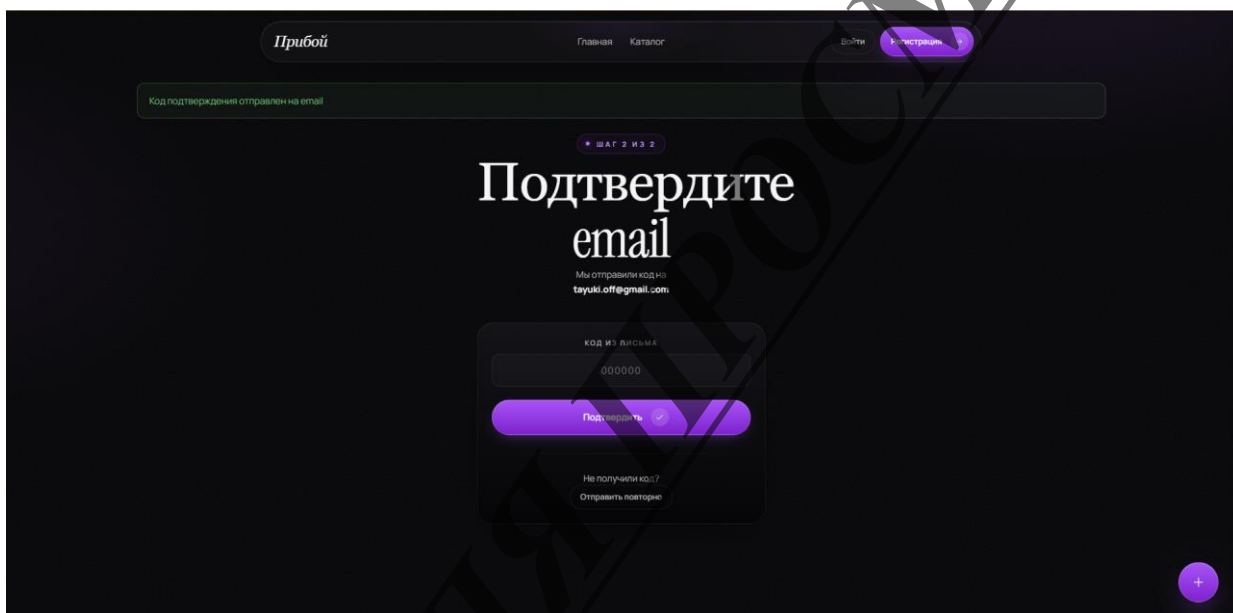


Рисунок 16 - Страница подтверждения e-mail при регистрации

ЗАКЛЮЧЕНИЕ

В рамках ВКР разработано веб-приложение для бара «Прибой», объединяющее каталог меню, предзаказ на вынос и бронирование столов с кальянами. Цель достигнута, задачи выполнены.

Технологии. Выбран стек Django, HTMX, Alpine.js, Tailwind CSS, тема админки Jazzmin. По сравнению с React/Vue стек даёт меньшую трудоёмкость разработки и поддержки при хорошем клиентском опыте.

База данных. 15 сущностей, ER-диаграмма, 3НФ, все миграции Django.

Интерфейс. Модальное окно с вкладками «Корзина» и «Бронирование». Состояние - в глобальных хранилищах Alpine.js. Отправка заказов и брони - HTMX-запросами с возвратом HTML-фрагментов, уведомления и редирект через NX-Trigger и NX-Location.

Админка. Jazzmin (тёмная тема, фиолетовый акцент). Настроены ModelAdmin: фильтры, поиск, inline-формы, массовые действия, иерархия по датам.

Поиск. Клиент-серверный с дебаунсом 300 мс. В перспективе - полнотекстовый поиск PostgreSQL (GIN, pg_trgm).

Результаты. 15 моделей, 5 приложений Django, 20+ HTML-шаблонов, 3 хранилища Alpine.js, 10+ HTMX-событий. Приложение полностью работоспособно.

Перспективы. PostgreSQL с полнотекстом, email-уведомления, эквайринг, программа лояльности, интеграция с мессенджерами, в будущем - PWA/мобильное приложение и публичный API.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Карпов Б. И. Базы данных: модели, разработка, реализация / Б. И. Карпов. - 2-е изд., перераб. - СПб.: Питер, 2023. - 320 с.
2. Гарсиа-Молина Г. Системы баз данных. Полный курс / Г. Гарсиа-Молина, Дж. Ульман, Дж. Уидом; пер. с англ. - М.: Вильямс, 2024. - 1100 с.
3. Кренке Д. Теория и практика построения баз данных / Д. Кренке; пер. с англ. - 11-е изд. - СПб.: Питер, 2023. - 880 с.
4. Коннолли Т. Базы данных. Проектирование, реализация и сопровождение / Т. Коннолли, К. Бегг; пер. с англ. - 7-е изд. - М.: Вильямс, 2023. - 1450 с.
5. Лутц М. Изучаем Python / М. Лутц; пер. с англ. - 6-е изд. - М.: Диалектика, 2025. - 900 с.
6. Гриффитс Д. Head First. Изучаем программирование на Python / Д. Гриффитс, П. Барри; пер. с англ. - 3-е изд. - М.: Эксмо, 2023. - 700 с.
7. Билл Л. Python. К вершинам мастерства / Л. Билл; пер. с англ. - 2-е изд. - М.: ДМК Пресс, 2024. - 480 с.
8. Рамальо Л. Python. Карманный справочник / Л. Рамальо; пер. с англ. - 6-е изд. - М.: Диалектика, 2023. - 360 с.
9. Доусон М. Программируем на Python / М. Доусон; пер. с англ. - 5-е изд. - СПб.: Питер, 2024. - 450 с.
10. Гринфелд Д. Two Scoops of Django 5.x: Best Practices for the Django Web Framework / Д. Р. Гринфелд, О. Рой-Гринфелд. - Two Scoops Press, 2025. - 560 р.
11. Винсент В. С. Django for Professionals: Production websites with Python and Django / В. С. Винсент. - 5-е изд. - Wisepunk Press, 2025. - 320 р.
12. Васкес Бронштейн А. Веб-разработка с Python: Django, Flask и базы данных / А. Васкес Бронштейн. - 3-е изд. - М.: ДМК Пресс, 2024. - 420 с.

- 13.Гринберг Д. Django. Разработка веб-приложений на Python / Д. Гринберг; пер. с англ. - 4-е изд. - М.: Эксмо, 2025. - 550 с.
- 14.Гринфилд Д. Flask: создание веб-приложений на Python / Д. Гринфилд; пер. с англ. - 4-е изд. - М.: ДМК Пресс, 2024. - 340 с.
- 15.Норман Д. А. Дизайн привычных вещей / Д. А. Норман; пер. с англ. - М.: Манн, Иванов и Фербер, 2024. - 400 с.
- 16.Круг С. Веб-дизайн. Не заставляйте меня думать / С. Круг; пер. с англ. - 4-е изд. - СПб.: Питер, 2023. - 280 с.
- 17.Купер А. Об интерфейсе. Основы проектирования взаимодействия / А. Купер, Р. Райман, Д. Кронин; пер. с англ. - 3-е изд. - СПб.: Питер, 2024. - 760 с.
- 18.Хоган Б. HTML5 и CSS3. Веб-разработка по стандартам / Б. Хоган; пер. с англ. - 4-е изд. - М.: Питер, 2024. - 300 с.
- 19.Флэнаган Д. JavaScript. Подробное руководство / Д. Флэнаган; пер. с англ. - 8-е изд. - М.: Диалектика, 2023. - 1200 с.
- 20.Грант К. CSS в действии / К. Грант; пер. с англ. - 3-е изд. - СПб.: Питер, 2024. - 480 с.
- 21.Браун Т. Tailwind CSS: быстрая адаптивная вёрстка / Т. Браун; пер. с англ. - 2-е изд. - М.: ДМК Пресс, 2025. - 280 с.
- 22.Дуглас К. PostgreSQL. Основы языка SQL / К. Дуглас, С. Дуглас; пер. с англ. - 4-е изд. - СПб.: БХВ-Петербург, 2024. - 520 с.
- 23.Карп Н. Г. Современные веб-технологии: микросервисы, контейнеризация и CI/CD / Н. Г. Карп, А. В. Белов. - М.: ДМК Пресс, 2024. - 480 с.
- 24.Браун К. Безопасность веб-приложений на Python: защита от уязвимостей / К. Браун; пер. с англ. - 3-е изд. - СПб.: БХВ-Петербург, 2025. - 400 с.